Lecture 1 revision sheet

Introduction to modelling with mathematical, computational and data-science techniques

This sheet lists the definitions, formulas and results from Lecture 1, without derivations or discussion. These are in the full notes (`introduction.ipynb`, homework and supplementary notebooks); the grey pointers give the section. → L*n* marks material that later lectures build on. [*] marks material that is optional in the notes.

Key messages

1. A model is a **mapping** from inputs (initial conditions, parameters) to outputs. *All models are wrong, but some are useful.* The question to ask is "Is it good enough for this application?", not "Is it true?".
2. Going from a real system to a computer solution takes several modelling steps (conceptual → mathematical → numerical → code). Every step makes assumptions and approximations.
3. Continuous models (ODEs, PDEs) must be **discretised** to run on a computer, and discretisation introduces error.
4. Simple discrete maps can behave in very complicated ways, e.g. oscillation, period doubling, chaos and blow-up.
5. **Taylor series** are the basic tool for building numerical methods and analysing their errors.
6. Forward Euler applied to the logistic ODE *is* the logistic map with $r = \alpha\Delta t$. Its stability depends on the product of a physical parameter and a numerical one: **you cannot choose the numerics without knowing the physical parameters.**
7. Errors accumulate. Convergence plots (log–log, slope = order) are how we check a code against theory (*code verification*).

1 Modelling vocabulary

Full notes: *Some terminology…; Computational Science vs Data Science; supplementary reading notebook*

Model	Something that mimics a (usually complex or poorly understood) system or process. Mathematically, a function or mapping from inputs to outputs. "Model" means different things in different fields, so agree on terminology with collaborators.
Model hierarchy	conceptual → scientific → mathematical → numerical/computer model. A *surrogate model* is a cheap approximation built from input–output data, often from runs of an expensive computer model.
Simulation	*Running* a model. You build a model and then run it (or "run a simulation").
Mechanistic (process-based)	Built from first principles or physical laws. Focus is on causality, and such models can extrapolate ("what if?").
Data-driven (empirical)	Learned from patterns in data. Focus is on correlation and predictive accuracy, and the model may be a black box. Machine learning is mostly here, but is increasingly used inside computational science (surrogates, physics-informed ML).
Discretisation	Turning a continuous model (ODE/PDE) into a finite, discrete one that a computer can solve. The *continuum assumption* treats a system that is discrete at very small scales (e.g. molecules) as continuous.
Sensitivity	How outputs respond to uncertain inputs: initial conditions, physical parameters, and *numerical* parameters such as Δt.
V&V	**Code verification** (is the code solving the equations correctly?), **solution verification** (is the numerical error acceptable for this run?) and **model validation** (are these the right equations for reality?). Named here and defined in L4. **→ used in L4**

2 Discrete maps: the logistic map

Full notes: *Example – the logistic map; homework: classical form, perturbations, Ricker, Fibonacci*

Two forms

	Map $x_{n+1} = F(x_n)$	Notes
Classical	$x_{n+1} = r\,x_n(1 - x_n)$	$x_n \in [0, 1]$ is the population as a fraction of the maximum. For small x_n, $x_{n+1} \approx r x_n$.
Lecture form	$x_{n+1} = x_n + r\,x_n(1 - x_n)$	The change $\Delta x = r x_n(1 - x_n)$. For small x_n, $x_{n+1}/x_n \approx 1 + r$, so r is the growth rate per generation. $x_n > 1$ is allowed (the population then falls).

- **Equivalence:** substituting $x_n = \frac{1+r}{r}\,y_n$ in the lecture form gives the classical map with parameter $1 + r$. So the lecture form at r behaves like the classical form at $r + 1$.

- Without the $(1-x_n)$ factor: $x_{n+1} = rx_n \Rightarrow x_n = r^n x_0$, i.e. unbounded exponential growth for $r > 1$. The factor $(1-x_n)$ makes the growth rate fall linearly to zero at the maximum population.
- Roots of the lecture-form right-hand side $x(1+r-rx)$: $x = 0$ and $x = (1+r)/r$.

Fixed points and their stability

→ **used in L3, L6**

- **Fixed point:** $x^* = F(x^*)$, where the graph of F crosses the line $y = x$. Lecture form: $x^* = 0$ and $x^* = 1$. Classical form: $x^* = 0$ and $x^* = 1 - 1/r$.
- **Linear stability** (first-order Taylor expansion of F about x^*): writing $x_n = x^* + \varepsilon_n$,

$$\varepsilon_{n+1} \approx F'(x^*)\,\varepsilon_n.$$

  The fixed point is stable if $|F'(x^*)| < 1$ and unstable if $|F'(x^*)| > 1$. If $F'(x^*) > 0$ the approach is monotonic; if $F'(x^*) < 0$ the iterates oscillate either side.
- Lecture form: $F'(x) = 1 + r - 2rx$, so $F'(1) = 1 - r$ and $F'(0) = 1 + r > 1$ ($x^* = 0$ is always unstable for $r > 0$). Classical form: $F'(1 - 1/r) = 2 - r$, and $x^* = 0$ is stable only for $r < 1$.
- **Cobweb plot:** start at (x_0, x_0) on $y = x$, go vertically to the curve $y = F(x)$, go horizontally back to $y = x$, and repeat.

Behaviour as r increases

Lecture form r	Classical r	Long-term behaviour (starting from $0 < x_0 < (1+r)/r$)
$0 < r \le 1$	$1 < r \le 2$	Monotonic convergence to $x^* = 1$ ($0 \le F'(1) < 1$).
$1 < r < 2$	$2 < r < 3$	Oscillatory convergence to $x^* = 1$. It gets slower as $r \to 2$.
$2 < r < \sqrt{6} \approx 2.449$	$3 < r < 3.449$	Fixed point unstable, giving a period-2 ("mode 2") cycle.
$2.449 < r < 2.544$	$3.449 < r < 3.544$	Period 4, followed by a period-doubling cascade (8, 16, …).
$\approx 2.57 < r \le 3$	$\approx 3.57 < r \le 4$	Mostly chaotic but bounded, with some periodic windows. At $r = 3$ (classical 4) the iterates wander over the whole interval.
$r > 3$	$r > 4$	Blow-up: the maximum of F, $(1+r)^2/(4r)$, exceeds $(1+r)/r$, so iterates leave the interval and run off to $-\infty$.

- **Bifurcation diagram:** long-term values plotted against r. Iterate many times (a "spin-up"), then plot the last values for each r. Vectorise over an array of r values.
- **Sensitivity to initial conditions depends on r.** For a stable fixed point, perturbations decay. In the chaotic regime a perturbation of 0.01 grows to $\mathcal{O}(1)$ within a few iterations.
- **Mathematical stability of a fixed point ≠ numerical stability of a simulation.** Blow-up is the numerical kind.
- Complex behaviour may be *real* (the system really is complex) or *spurious* (an artefact of a bad parameter choice, see §7).

Other recurrences in the homework [*]

Ricker map $x_{n+1} = x_n\, e^{\,r(1-x_n)}$. It has no upper limit on r before it blows up or goes negative.

Fibonacci $F_{i+1} = F_i + F_{i-1}$, with $F_0 = 0$, $F_1 = 1$. Python integers are exact, so F_{1000} is exact. → **used in L3, L4**

3 Taylor series

Full notes: *Taylor series* → **used in L4–L7**

$$f(x) = f(x_0) + (x-x_0)f'(x_0) + \frac{(x-x_0)^2}{2!}f''(x_0) + \frac{(x-x_0)^3}{3!}f'''(x_0) + \mathcal{O}\big((x-x_0)^4\big)$$

$$f(x_0+h) = f(x_0) + hf'(x_0) + \frac{h^2}{2!}f''(x_0) + \frac{h^3}{3!}f'''(x_0) + \mathcal{O}(h^4) \qquad (h \text{ is also written } \Delta x,\ \delta x)$$

- The series approximates f near the **expansion point** x_0, using only the value and derivatives of f at x_0. Adding more terms makes the approximation valid further from x_0.
- **Constant approximation**: $f(x) \approx f(x_0)$ (cf. "persistence" forecasting). Its accuracy is governed by f'. **Linearisation**: $f(x) \approx f(x_0) + (x-x_0)f'(x_0)$. Its accuracy is governed by f''.
- Examples: $e^x = \sum_{k\ge 0} \frac{x^k}{k!}$ and $\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \cdots$ (expansions about 0).
- **Truncation:** keeping the terms up to h^{N-1} leaves an error of $\mathcal{O}(h^N)$ as $h \to 0$. This is a straight line of slope N on a log–log plot of error against h.

4 Big-O notation [*]

Full notes: *Aside: Big O notation* → used in L2, L4, L6, L7

- **Small-parameter limit (errors):** $g(h) = \mathcal{O}(h^p)$ as $h \to 0$ means that there is a constant C with $|g(h)| \leq C\,h^p$ for all sufficiently small h. The *lowest* power dominates.
  Example: $3y^2 - y^3 + 9y^4 = \mathcal{O}(y^2)$ as $y \to 0$ (take $C = 13$ for $|y| \leq 1$), and $= \mathcal{O}(y^4)$ as $y \to \infty$.
- **Large-parameter limit (cost):** an algorithm with complexity $\mathcal{O}(n^p)$ in problem size n has cost that grows like n^p for large n, where the *highest* power dominates. For example, $\mathcal{O}(n^2)$ means that doubling n multiplies the cost by 4. $\mathcal{O}(n \log n)$ grows faster than linear but slower than quadratic.
- **Error of order p:** if the error is $\mathcal{O}(\Delta t^p)$, halving Δt divides the error by about 2^p.
- **Asymptotic regime:** with error $\approx 10^{-5}\Delta t + 10^5 \Delta t^2$, the error looks second order at large Δt but is $\mathcal{O}(\Delta t)$. The linear term dominates once $\Delta t < 10^{-10}$. The order is only observed once you are "in the asymptotic limit".

5 Errors

Full notes: *Truncation error; vs Round-off errors* → used in L2, L4, L6, L7

- **Truncation error:** from replacing an infinite process (e.g. a Taylor series) by a finite one.
- **Round-off error:** from the computer's finite-precision (floating-point) representation of real numbers.
- **Errors accumulate** from step to step in a time-stepping method (see §6).

6 ODEs and the forward Euler method

Full notes: *Using Taylor series to solve an ODE numerically* → used in L3, L4, L6, L7

- **ODE:** an equation for an unknown function of *one* independent variable, $F(t, y, y', \ldots, y^{(n)}) = 0$, where t is the independent and $y(t)$ the dependent variable. A vector $\boldsymbol{y}$ gives a *system* of ODEs. More than one independent variable gives a *PDE*.
- **First-order initial value problem:** $y' = f(t, y)$ with $y(t_0) = y_0$. A first-order ODE needs one piece of auxiliary data, such as an initial condition.

Forward (explicit) Euler

From $y(t + \Delta t) = y(t) + \Delta t\, y'(t) + \mathcal{O}(\Delta t^2)$, drop the $\mathcal{O}(\Delta t^2)$ terms and use $y'_n = f(t_n, y_n)$:

$$y_{n+1} = y_n + \Delta t\, f(t_n, y_n) \qquad \Longleftrightarrow \qquad \frac{y_{n+1} - y_n}{\Delta t} = f(t_n, y_n).$$

- **One step** (starting from the exact solution) makes an error of $\mathcal{O}(\Delta t^2)$, the *local* error.
- Reaching a fixed time T takes $T/\Delta t$ steps, so the **global error** at T is $\mathcal{O}(\Delta t^2) \times T/\Delta t = \mathcal{O}(\Delta t)$. Forward Euler is **first order**.
- Each step starts from the previous *approximate* value and uses the slope there, so errors accumulate.

Note for L6: there the *local truncation error* is defined per unit step, using the second form above. For forward Euler it is $\mathcal{O}(\Delta t)$, and local error $= \Delta t \times$ LTE.

```
for n in range(len(t)-1):  x[n+1] = x[n] + dt*f(t[n], x[n])
```
(the whole solver; the rest is set-up and plotting)

Model problems with exact solutions (used to test code)

Problem	Exact solution	Notes
$y' = y,\ y(0) = 1$	$y = e^t$	Forward Euler undershoots here, and the error grows step by step.
Relaxation [*]: $C' = K(C_r - C)$	$C = C_r + (C_0 - C_r)e^{-Kt}$	$C \to C_r$ as $t \to \infty$. K has units of 1/time. Radioactive decay ($C_r = 0$): $K = \ln 2 / t_{1/2}$. → used in L6
Logistic ODE: $x' = \alpha x(1 - x)$	$x(t) = \dfrac{x_0}{x_0 + (1 - x_0)e^{-\alpha t}}$	Found by separation of variables, using $\frac{1}{x(1-x)} = \frac{1}{x} + \frac{1}{1-x}$. $x \to 1$ as $t \to \infty$ (for $x_0 > 0$). The solution is the logistic (sigmoid) function $\sigma(z) = 1/(1 + e^{-z})$ evaluated at $z = \alpha t + \ln \frac{x_0}{1-x_0}$. → used in L4, L8

SciPy's `solve_ivp` (RK45: higher order, with adaptive Δt) gives errors orders of magnitude smaller than forward Euler. → used in L3, L7

7 Forward Euler for the logistic ODE is the logistic map

Full notes: *Back to the logistic problem* → **used in L6**

Forward Euler on $x' = \alpha x(1-x)$ gives $x_{n+1} = x_n + \alpha\Delta t\, x_n(1-x_n)$, which is the lecture-form logistic map with

$$r = \alpha\,\Delta t \quad \text{(physical parameter} \times \text{numerical parameter)}.$$

So the table in §2 applies to the *numerical* solution. The ODE itself always converges smoothly to $x = 1$, so everything past $\alpha\Delta t = 1$ is a numerical artefact. Lecture example, with $\alpha = 0.95$:

Δt	1	2	3	3.16
$\alpha\Delta t$	0.95	1.9	2.85	3.002
Numerical solution	monotonic, correct	oscillates, then converges	irregular (chaotic), bounded	blows up

The numerical modeller cannot choose the numerics (here Δt) independently of the physical parameters (here α). If α changes, the stable choice of Δt changes.

Looking ahead (L6). Linearise about $x = 1$: $f(x) = \alpha x(1-x)$ has $f'(1) = -\alpha =: \lambda$. The perturbation multiplier of §2, $1 - r = 1 - \alpha\Delta t$, is then $1 + \lambda\Delta t$, the forward Euler *amplification factor*. The fixed point is stable when $|1 + \lambda\Delta t| < 1 \iff 0 < \alpha\Delta t < 2$, which is exactly the $r = 2$ threshold. L6 uses the same condition for relaxation ($\lambda = -K$): $0 < K\Delta t < 2$.

8 Convergence analysis (code verification)

Full notes: *Comparing numerical and analytical solutions quantitatively* → **used in L4–L7**

1. Choose a problem with an exact solution and a fixed end time T. Pick T before the solution reaches steady state, because the steady state hides earlier errors (the lecture uses $T = 5$).
2. Run with a sequence of step sizes (e.g. halving Δt). Make sure every run ends exactly at T.
3. Record error $= |x_{\text{num}}(T) - x_{\text{exact}}(T)|$.
4. Plot on log–log axes. If error $\approx C\Delta t^p$ then $\log(\text{error}) \approx \log C + p\log\Delta t$: a straight line whose **slope is the order** p. Fit it with `np.polyfit` on the logs, leaving out the large-Δt (pre-asymptotic or unstable) points.
5. Compare with theory. Forward Euler on the logistic ODE gives slope ≈ 1.00. A match is **code verification**.